N052 로드밸런서로 무중단 배포 구성하기_해설서

Chapter01

/강의정리 챕터 01. 로드밸런서로 무중단 배포 구성하기

1. 강의의 목적

이번 강의의 핵심은 AI 서비스를 운영하면서 새 버전을 배포하더라도 사용자가 서비스를 계속 이용하고, 기존 대화 내용도 유지되도록 만드는 방법을 배우는 것입니다.

지금까지는 AI 에이전트를 만들고 실행하는 방법을 배웠다면, 이제는 실제 사용자가 이용하는 서비스를 안정적으로 운영하는 방법으로 넘어갑니다.

예를 들어 직접 준비하고 계신 MSM(My-Story-Maker) 서비스를 생각해 보겠습니다. 사용자가 자신의 일기를 입력하고 AI와 대화하면서 자서전을 만드는 중에 서비스가 업데이트되더라도, 사용자가 오류를 겪거나 지금까지 작성한 이야기가 사라져서는 안 됩니다.

이번 강의에서는 이러한 문제를 해결하기 위해 로드밸런서, 무중단 배포, 데이터베이스를 함께 활용합니다.

2. 중학생도 이해할 수 있는 핵심 개념

People in supermarket grocery store.

① 가용성(Availability)

서비스가 정상적으로 작동하여 사용자가 이용할 수 있는 상태입니다.

- 가용성이 높다: 언제 접속해도 대부분 정상적으로 이용할 수 있습니다.
- 가용성이 낮다: 서비스가 자주 멈추거나 접속되지 않습니다.

② 단일 장애점(SPOF)

서비스에서 특정 요소 하나가 고장 나면 전체 서비스가 멈추는 지점입니다.

예를 들어 서버가 한 대뿐이라면 그 서버가 멈췄을 때 사용자는 서비스를 이용할 수 없습니다.

🔄Automated Failover: Designing for Failure, Recovering Without Drama | by Chaos To Clarity | Medium

③ 이중화(Redundancy)

문제가 생겨도 서비스를 계속 제공할 수 있도록 서버나 구성 요소를 여러 개 준비하는 것입니다.

서버 A가 고장 나더라도 서버 B가 정상적으로 요청을 처리하도록 구성할 수 있습니다.

How to OVER Engineer a Website // What is a Tech Stack? - DEV Community

④ 무상태(Stateless)

서버 자체에 사용자 대화의 유일한 사본을 보관하지 않고, 필요한 상태를 외부 저장소에서 가져오는 구조입니다.

서버 A에서 대화를 나눈 뒤 서버 B로 연결되더라도 외부 데이터베이스에서 대화 내용을 읽으면 대화를 이어갈 수 있습니다.

쉽게 말하면 다음과 같습니다.

- 가용성은 얼마나 계속 이용할 수 있는가
- 단일 장애점은 어디 하나가 고장 나면 전체가 멈추는가
- 이중화는 고장에 대비해 여분을 준비하는가
- 무상태는 사용자 정보가 특정 서버에만 묶여 있지 않은가

3. 무중단 배포는 어떻게 작동할까요?

무중단 배포란 새로운 버전으로 교체하는 동안에도 사용자가 서비스를 이용할 수 있도록 배포하는 방식입니다.

배포 전환 과정

사용자 요청

Nginx 로드밸런서

요청을 어느 서버로 보낼지 결정

Blue

서버 A

현재 운영 중인 버전

Green

서버 B

새 버전을 준비하고 테스트

예시: 새 버전의 서버 B가 정상인지 확인한 후 Nginx의 요청 경로를 B로 변경합니다. 문제가 발생하면 다시 A로 돌릴 수 있습니다.

대표적인 배포 방식 3가지

방식	작동 원리	장점과 특징
롤링(Rolling)	서버를 조금씩 교체	자원을 절약할 수 있지만 교체 중 구버전과 신버전이 함께 운영됩니다.
블루그린(Blue-Green)	기존 환경과 새 환경을 함께 준비한 뒤 전환	전환과 롤백이 비교적 단순하지만 두 환경을 준비할 자원이 필요합니다.
카나리(Canary)	일부 사용자에게 먼저 새 버전을 제공	실제 사용 환경에서 문제를 확인하며 점진적으로 확대할 수 있습니다.

이번 강의에서 직접 실습할 핵심은 블루그린 배포입니다.

4. Nginx와 로드밸런서는 무슨 일을 할까요?

로드밸런서는 들어오는 요청을 여러 서버에 적절하게 분배하는 역할을 합니다.

Nginx는 웹 서버로도 쓰이지만, 이번 실습에서는 사용자와 에이전트 서버 사이에서 요청을 전달하는 리버스 프록시이자 로드밸런서로 사용합니다.

- 리버스 프록시: 사용자의 요청을 대신 받아 실제 서버에 전달합니다.
- 로드밸런서: 여러 서버 중 요청을 처리할 대상을 선택합니다.
- 헬스체크(Health Check): 서버가 정상적으로 작동하는지 확인합니다.
- 그레이스풀 셧다운(Graceful Shutdown): 서버를 종료할 때 진행 중인 요청이 가능한 한 정상적으로 끝나도록 기다립니다.
- 롤백(Rollback): 새 버전에 문제가 생기면 이전 버전으로 되돌립니다.

여기서 주의할 점은 로드밸런서가 있다고 해서 무조건 무중단 배포가 보장되는 것은 아니라는 사실입니다. 서버 준비 상태, 요청 처리 중 종료, 데이터베이스 호환성까지 함께 고려해야 합니다.

5. Docker와 연결해서 이해하기

이번 강의는 Docker를 이미 배운 상태에서 시작합니다. 앞에서 배운 개념과 연결하면 다음과 같습니다.

1. 코드

AI 에이전트의 프로그램을 작성합니다.

2. Docker 이미지

   코드를 실행하는 데 필요한 프로그램과 라이브러리를 묶습니다.

3. 컨테이너

   이미지를 실제로 실행한 인스턴스입니다. 같은 서비스의 컨테이너를 두 개 실행할 수 있습니다.

4. Nginx

   두 컨테이너 앞에서 요청을 전달합니다.

5. 배포 전환

   Nginx가 새 버전의 컨테이너로 요청을 보내도록 변경합니다.

쉽게 비유하면 Docker 이미지는 제품의 설계도와 포장 세트이고, 컨테이너는 그 제품을 실제로 작동시킨 것입니다. Nginx는 고객을 어느 창구로 안내할지 결정하는 안내 직원입니다.

6. 가장 중요한 문제: 대화 내용은 어디에 저장할까요?

무중단 배포에서 놓치기 쉬운 부분입니다.

예를 들어 사용자가 서버 A에서 AI 친구와 10분간 대화를 나눴다고 가정해 보겠습니다.

- 대화 내용이 서버 A의 메모리에만 있다면, 서버 B로 전환했을 때 대화가 사라진 것처럼 보일 수 있습니다.
- 대화 내용이 외부 데이터베이스에 저장되어 있다면, 서버 B도 같은 데이터를 읽어 대화를 이어갈 수 있습니다.

이번 강의에서는 Supabase에 데이터와 익명 로그인을 연결합니다.

사용자

Nginx

요청 전달

컨테이너 A

컨테이너 B

Supabase 데이터베이스

대화 내용과 사용자 식별 정보 저장

익명 로그인은 사용자가 별도의 회원가입 절차를 거치지 않아도 서비스에서 사용자를 구분할 수 있도록 하는 방식입니다. 단, 익명 사용자 식별 정보와 대화 데이터의 연결, 접근 권한

정책을 올바르게 설정해야 합니다.

핵심은 서버를 교체해도 사용자 데이터까지 함께 교체하거나 사라지게 만들지 않는 것입니다.

7. 직접 실습할 때 무엇을 확인해야 할까요?

이번 강의의 목표는 개념을 이해하는 데 그치지 않고 실제로 작동하는지 확인하는 것입니다.

- [ ] Nginx와 에이전트 서비스 컨테이너 두 개를 실행합니다.
- [ ] Nginx 설정에서 현재 요청을 전달하는 서버를 확인합니다.
- [ ] 설정 한 줄을 변경하여 블루그린 전환을 수행합니다.
- [ ] 새 버전에서 문제가 발생하면 이전 버전으로 롤백합니다.
- [ ] 전환하는 동안 요청이 얼마나 실패하는지 측정합니다.
- [ ] 서버 하나를 일부러 멈추고 Nginx가 어떻게 반응하는지 확인합니다.
- [ ] Supabase에 대화를 저장하고 서버를 전환한 뒤에도 대화가 이어지는지 확인합니다.

실습에서는 단순히 화면에 성공이라고 표시되는지만 보지 말고, 요청 실패 여부와 대화 데이터의 지속 여부를 각각 확인해야 합니다.

8. AI 서비스 개발자와 1인 창업가에게 왜 중요할까요?

AI 서비스를 만드는 것과 AI 서비스를 사업으로 운영하는 것은 다릅니다.

서비스를 공개한 뒤에는 사용자가 언제 접속할지 알 수 없고, 업데이트 중에도 요청이 들어올 수 있습니다. 따라서 배포 과정에서 서비스가 멈추거나 사용자 데이터가 사라지면 신뢰와 사업 기회가 함께 손상될 수 있습니다.

앞으로 준비하실 AI 컨설팅 서비스와 MSM에도 다음 원칙을 적용할 수 있습니다.

- 서비스 안정성: 업데이트 중에도 이용할 수 있도록 배포 구조를 설계합니다.
- 복구 가능성: 새 버전에 문제가 생겼을 때 이전 버전으로 되돌릴 수 있게 합니다.
- 데이터 지속성: 사용자 기록을 특정 서버에만 의존하지 않도록 합니다.
- 검증 가능한 운영: 감으로 판단하지 않고 요청 실패율과 실제 데이터로 확인합니다.

처음부터 대규모 인프라를 구축할 필요는 없습니다. 로컬에서 두 개의 컨테이너를 전환해 보는 이번 실습만으로도 운영 구조의 기본 원리를 익힐 수 있습니다.

9. 반드시 기억해야 할 내용

1. 가용성은 서비스를 정상적으로 이용할 수 있는 정도입니다.
2. 이중화는 고장에 대비해 대체할 구성 요소를 준비하는 것입니다.
3. 무중단 배포는 새 버전으로 바꾸는 동안에도 사용자 요청을 계속 처리하도록 하는 것입니다.
4. Nginx 로드밸런서는 요청을 전달할 서버를 선택하고, 설정에 따라 서버 전환을 지원합니다.
5. 헬스체크와 그레이스풀 셧다운은 배포 과정의 오류와 진행 중인 요청의 중단을 줄이는 데 도움이 됩니다.
6. 외부 데이터베이스는 서버가 바뀌어도 대화와 같은 사용자 데이터를 유지하는 데 중요합니다.
7. 로드밸런서만 설치했다고 무중단 배포가 완성되는 것은 아닙니다. 서버 상태, 데이터 호환성, 실제 요청 실패 여부까지 확인해야 합니다.

10. 한 문장 요약

무중단 배포란 서버를 단순히 새 버전으로 교체하는 것이 아니라, 요청을 안전하게 전환하고 문제가 생기면 복구하며 사용자 데이터까지 계속 유지하도록 설계하는 것입니다.

시작 질문에 대한 예시 답변

교안 첫 번째 질문에 직접 답해야 한다면 다음과 같이 작성하실 수 있습니다.

에이전트 서비스가 새 버전으로 바뀌는 순간, 서버가 잠시 중단되어 사용자가 오류 메시지를 보거나 요청이 실패할 수 있습니다. 서버가 다시 실행되더라도 기존 대화 내용이 서버 내부의 메모리에만 저장되어 있다면 대화가 사라질 수도 있습니다. 따라서 새 버전을 배포하기 전에 다른 서버를 준비하고, 로드밸런서로 요청을 전환하며, 대화 데이터는 외부 데이터베이스에 저장해야 한다고 생각합니다.

이 답변은 이번 강의에서 다룰 서비스 가용성, 서버 전환, 데이터 지속성이라는 세 가지 문제를 모두 짚고 있습니다.

Chapter02

/강의정리 챕터 02. 가용성을 숫자로 말하기

1. 강의의 목적

이번 강의의 핵심은 AI 서비스를 얼마나 안정적으로 운영할 것인지 숫자로 정하고, 서버에 문제가 생겨도 서비스를 계속 제공할 수 있도록 구조를 설계하는 것입니다.

챕터 01에서는 Nginx를 이용해 서버 두 벌 사이에서 요청을 전환하는 무중단 배포를 배웠습니다. 이번 챕터에서는 그보다 한 단계 더 나아가 다음 질문에 답합니다.

- 서버가 멈추면 사용자의 요청은 어떻게 될까요?
- 서버를 두 대 운영하면 정말 안전할까요?
- 사용자가 다른 서버로 연결되어도 대화와 로그인 정보가 유지되려면 어떻게 해야 할까요?
- 새 버전을 배포한 뒤 문제가 생기면 어떻게 이전 버전으로 돌아갈까요?

이 내용은 앞으로 만드실 MSM(My-Story-Maker)이나 AI 컨설팅 서비스를 실제 사용자에게 제공할 때 반드시 필요한 운영 원칙입니다.

2. 중학생도 이해할 수 있는 핵심 개념

2-1. 가용성(Availability): 서비스가 얼마나 자주 정상적으로 작동할까요?

가용성은 전체 시간 중 서비스를 정상적으로 사용할 수 있었던 시간의 비율입니다.

1년을 기준으로 계산한 허용 중단 시간

99%

투 나인

약 3일 15시간 36분

99.9%

쓰리 나인

약 8시간 46분

99.99%

포 나인

약 52분 34초

99.999%

파이브 나인

약 5분 15초

1년 365일을 기준으로 한 이론적 중단 시간입니다. 실제 서비스의 가용성 측정 기준과 계약 조건에 따라 계산 방식은 달라질 수 있습니다.

99%라는 숫자는 굉장히 높아 보입니다. 하지만 1년 동안 약 3.65일이나 서비스를 사용할 수 없다는 뜻입니다.

가용성이 99.9%에서 99.99%로 높아지면 허용 중단 시간은 약 8시간 46분에서 약 53분으로 줄어듭니다.

여기서 중요한 점은 가용성을 높일수록 시스템이 더 복잡해지고 운영 비용도 커질 수 있다는 것입니다. 따라서 모든 서비스가 무조건 파이브 나인을 목표로 할 필요는 없습니다. 서비스의 중요도와 장애 비용에 맞게 목표를 정해야 합니다.

2-2. 단일 장애점(SPOF)과 이중화

シンプルおしゃれなサーバー｜AI百科「シンテリ」で調べる語句・イラスト

단일 장애점(SPOF)

서버가 한 대뿐인데 그 서버가 고장 나면 전체 서비스가 멈춥니다. 전체 서비스의 운명이 한 곳에 달려 있는 것입니다.

🔄Automated Failover: Designing for Failure, Recovering Without Drama | by Chaos To Clarity | Medium

이중화(Redundancy)

같은 역할을 하는 서버를 두 대 이상 준비합니다. 한 대에 문제가 생겨도 다른 서버가 요청을 처리할 수 있습니다.

그런데 서버 두 대를 준비하는 것만으로는 충분하지 않습니다.

두 서버가 같은 전원이나 네트워크 장비, 데이터센터에 의존한다면 공통된 장애가 발생했을 때 둘 다 멈출 수 있습니다. 따라서 중요한 서비스에서는 장애가 함께 발생하지 않도록 구성 요소를 분산하는 것도 고려해야 합니다.

2022년 판교 데이터센터 화재 사례가 보여 주는 교훈도 여기에 있습니다. 서버를 여러 대 준비하는 것뿐 아니라, 장애 발생 시 복구하는 관리 체계까지 함께 대비해야 합니다.

2-3. 장애에 대응하는 세 단계

고가용성 구조는 다음 세 단계로 이해하면 쉽습니다.

1단계. 이중화

고장에 대비해 대체 서버를 준비합니다.

2단계. 장애 감지

헬스체크 등을 통해 서버가 정상적으로 작동하는지 확인합니다.

3단계. 장애 조치(Failover)

서버에 문제가 생기면 대체 서버로 요청을 넘깁니다.

즉, 대체 서버가 있어도 고장을 알아채지 못하면 소용이 없습니다. 고장을 알아채더라도 자동으로 전환하지 못하면 서비스 중단이 길어질 수 있습니다.

2-4. 액티브-스탠바이와 액티브-액티브

구분	액티브-스탠바이	액티브-액티브
평상시	한 서버가 주로 처리하고 다른 서버는 대기	여러 서버가 함께 처리
서버 한 대 고장	대기 서버로 전환	남은 서버들이 요청을 처리
장점	구조가 비교적 단순함	평소에도 여러 서버의 자원을 활용
주의점	대기 자원이 놀 수 있음	부하 분산, 데이터 일관성 등을 더 세심하게 관리해야 함

액티브-액티브라고 해서 서버 한 대가 고장 나도 무조건 정상 운영되는 것은 아닙니다. 남은 서버가 늘어난 요청을 감당할 수 있어야 합니다.

3. 무상태(Stateless): 서버가 달라져도 기억은 유지되어야 합니다

이번 강의에서 특히 중요하게 다뤄야 할 개념입니다.

사용자가 AI 친구와 대화한다고 생각해 보겠습니다.

방식 A. 서버 메모리에 대화를 저장

서버 A의 메모리

사용자: 여행을 좋아합니다.

AI: 다음 여행은 어디로 가고 싶으세요?

다음 요청이 서버 B로 이동

서버 B의 메모리

이전 대화가 없음 → AI가 맥락을 잃을 수 있음

방식 B. 공용 데이터베이스에 대화를 저장

공용 데이터베이스

사용자의 대화와 캐릭터 정보 저장

서버 A

같은 데이터를 읽음

서버 B

같은 데이터를 읽음

무상태 서버란 요청을 처리하는 데 필요한 사용자 상태를 특정 서버의 메모리에만 의존하지 않는 서버입니다.

정확히는 서버가 아무것도 기억하지 않는다는 뜻은 아닙니다. 요청 처리에 필요한 상태를 외부 저장소에서 가져오거나, 요청에 포함된 정보로 확인할 수 있도록 설계한다는 의미입니다.

이번 강의에서 설명한 에이전트 서비스는 캐릭터, 대화, 기억을 인스턴스 메모리에 보관하고 있기 때문에 새 인스턴스로 바뀌면 기존 상태가 사라질 수 있습니다. 이를 데이터베이스로 옮기면 서버를 교체하거나 늘릴 때도 대화를 이어 갈 수 있습니다.

4. 데이터베이스의 복제와 백업은 다릅니다

데이터베이스도 고장에 대비해야 합니다.

마스터(Primary)

데이터 쓰기와 변경을 담당

변경된 데이터를 복제

복제본 A

복제본 B

마스터에 장애가 발생하면 적절한 복제본을 새 마스터로 승격할 수 있습니다. 다만 복제 지연이나 장애 조치 정책에 따라 일부 최신 변경 사항이 반영되지 않았을 가능성도 고려해야 합니다.

여기서 꼭 구분할 개념이 있습니다.

- 복제(Replication): 데이터를 여러 데이터베이스에 복사하여 가용성과 읽기 처리 능력을 높입니다.
- 백업(Backup): 실수로 삭제하거나 데이터가 손상됐을 때 과거의 정상 상태로 복구할 수 있도록 별도로 보관합니다.

예를 들어 실수로 대화 기록 1,000개를 삭제했다면, 복제본에도 삭제가 전파될 수 있습니다. 따라서 복제본만으로는 삭제 이전 상태를 복구할 수 없을 수 있습니다.

Supabase와 같은 관리형 서비스를 활용하면 데이터베이스 운영 부담을 줄일 수 있지만, 백업과 복구 정책, 접근 권한까지 신경 쓰지 않아도 된다는 뜻은 아닙니다.

5. 무중단 배포 방식 세 가지 비교

[서비스 배포 전략 구상하기] 무중단 배포 3가지 방식 (Rolling, Blue-Green, Canary)

① 롤링 배포

서버를 한 대씩 새 버전으로 교체합니다. 서버 자원을 절약할 수 있지만, 교체하는 동안 구버전과 신버전이 동시에 요청을 받을 수 있습니다.

System Design Codex | Saurabh Dashora | Substack

② 블루그린 배포

기존 환경과 새 환경을 각각 준비하고, 새 환경이 정상임을 확인한 뒤 요청을 전환합니다. 문제가 생기면 기존 환경으로 되돌리기 쉽지만 두 환경을 준비할 자원이 필요합니다.

MACHINE LEARNING MODEL DEPLOYMENT PATTERNS | by Gideon oboro | Medium

③ 카나리 배포

일부 요청만 새 버전으로 보내고 오류율이나 응답 시간 등을 확인한 뒤 점차 확대합니다. 문제를 조기에 발견하는 데 유리하지만 트래픽 비율을 관리하고 결과를 관찰해야 합니다.

세 방식의 공통점은 기존 서비스를 한꺼번에 중단하지 않고 새 버전으로 전환한다는 것입니다. 하지만 실제 무중단 여부는 배포 방식뿐 아니라 서버 준비 상태, 요청 처리 방식, 데이터베이스 호환성에도 달려 있습니다.

6. 배포할 때 반드시 지켜야 할 세 가지

조건	의미	지키지 않으면
헬스체크	새 서버가 실제로 요청을 받을 준비가 되었는지 확인	준비되지 않은 서버로 요청이 전달될 수 있음
그레이스풀 셧다운	기존 서버가 진행 중인 요청을 가능한 한 마친 뒤 종료	응답 도중 연결이 끊길 수 있음
DB 호환성	구버전과 신버전이 같은 데이터베이스를 함께 사용해도 정상 작동하도록 설계	배포 중 오류가 발생하거나 롤백한 구버전이 작동하지 않을 수 있음

특히 데이터베이스 구조를 변경할 때는 주의가 필요합니다.

예를 들어 기존 데이터베이스의 `user_name` 열을 새 버전에서 `display_name`으로 바꾼다고 생각해 보겠습니다. 새 버전은 새 열을 사용하지만 구버전은 여전히 기존 열을 찾습니다. 배포 중 두 버전이 함께 실행되면 오류가 생길 수 있습니다.

이를 예방하는 대표적인 방법이 확장-축소 패턴(Expand and Contract) 입니다.

1. 새 열을 추가하되 기존 열은 유지합니다.
2. 구버전과 신버전이 함께 작동할 수 있도록 코드를 배포합니다.
3. 기존 열을 사용하는 버전이 모두 종료되었는지 확인합니다.

4. 충분히 안전하다고 판단된 뒤 기존 열을 제거합니다.

데이터베이스 구조 변경도 서버 배포처럼 한 번에 바꾸지 않고, 호환되는 단계를 거치는 것입니다.

7. AI 서비스 개발과 1인 창업에 적용하기

앞으로 만드실 MSM을 예로 들어 보겠습니다.

상황	필요한 설계
사용자가 AI와 나눈 대화가 사라지면 안 됨	대화 기록을 공용 데이터베이스에 저장
사용자가 몰려 서버 한 대로 처리하기 어려움	서버 추가와 로드밸런싱 고려
서버 한 대가 고장 나도 서비스를 제공해야 함	이중화, 장애 감지, 자동 장애 조치
새 기능 배포 후 문제가 생김	헬스체크, 지표 확인, 롤백
데이터베이스 구조를 변경함	구버전과 신버전의 호환성을 고려한 마이그레이션
사용자 데이터가 실수로 삭제됨	별도 백업과 복구 절차 마련

초기 1인 서비스라면 처음부터 복잡한 다중 지역 인프라를 구성하기보다, 데이터 손실 방지와 복구 가능한 배포 구조부터 확보한 뒤 서비스 규모에 맞춰 고가용성을 높이는 접근이 현실적입니다.

8. 객관식 퀴즈 정답과 해설

정답: 2번. 로그인 정보를 서버 밖의 공용 저장소에 두어 서버를 무상태로 만든다.

이유는 로그인 정보가 특정 서버의 메모리에만 저장되어 있기 때문입니다. 로드밸런서가 다음 요청을 다른 서버로 전달하면 그 서버는 기존 로그인 정보를 알지 못합니다.

로그인 상태를 적절한 공용 저장소나 검증 가능한 토큰 등을 이용해 관리하면, 여러 서버가 동일한 사용자 인증 상태를 확인할 수 있습니다.

다른 선택지가 적절하지 않은 이유는 다음과 같습니다.

- 1번: 서버를 한 대로 줄이면 다른 서버로 이동하는 문제는 줄어들 수 있지만, 장애 대응 능력이 떨어집니다.
- 3번: 헬스체크는 서버의 정상 여부를 확인하는 기능이지, 로그인 정보를 공유하는 기능이 아닙니다.
- 4번: 블루그린 배포는 새 버전으로 전환하는 방법이며, 서버 간 로그인 정보 공유 문제 자체를 해결하지 않습니다.

9. 반드시 기억해야 할 내용

1. 가용성은 숫자로 관리합니다. 9가 하나 늘 때마다 허용 중단 시간은 대략 10분의 1로 줄어듭니다.
2. 이중화만으로 충분하지 않습니다. 장애 감지와 자동 장애 조치까지 필요합니다.
3. 무상태 서버는 서버 간 이동에 유리합니다. 로그인과 대화 상태를 특정 서버에만 의존하지 않도록 설계해야 합니다.
4. 복제와 백업은 다릅니다. 복제는 가용성에, 백업은 과거 데이터 복구에 중요한 역할을 합니다.
5. 배포 방식에는 각각 장단점이 있습니다. 롤링은 순차 교체, 블루그린은 환경 전환, 카나리는 일부 트래픽으로 검증하는 방식입니다.
6. 헬스체크, 그레이스풀 셧다운, DB 호환성은 무중단 배포의 기본 조건입니다.
7. 가용성 목표는 서비스의 중요도와 비용에 맞게 정해야 합니다. 모든 서비스가 파이브 나인을 필요로 하는 것은 아닙니다.

10. 한 문장 요약

고가용성 서비스란 서버를 여러 대 두는 것에서 끝나는 것이 아니라, 장애를 감지하고 자동으로 대응하며 서버가 바뀌어도 사용자 데이터와 서비스의 연속성을 유지하도록 설계한 시스템입니다.

Chapter03

/강의정리 챕터 03. 로드밸런서가 하는 일

1. 강의의 목적

이번 강의의 핵심은 로드밸런서가 사용자 요청을 여러 서버에 분배하고, 서버 장애와 배포 상황에 대응하는 방법을 이해하는 것입니다.

챕터 01에서는 Nginx를 이용한 무중단 배포를 배웠고, 챕터 02에서는 가용성, 이중화, 무상태 서버의 개념을 익혔습니다. 이번에는 실제로 요청을 분배하는 로드밸런서가 어떤 규칙으로 서버를 선택하고, 고장 난 서버를 어떻게 처리하는지 살펴봅니다.

이번 강의에서는 특히 다음 네 가지를 이해하는 것이 중요합니다.

- 어떤 서버로 요청을 보낼지 결정하는 분배 규칙
- 서버가 고장 났는지 알아내는 헬스체크
- 서버를 교체할 때 기존 요청을 안전하게 마무리하는 드레이닝

- 설정을 바꾸면서 서비스를 중단하지 않는 Nginx reload

2. 중학생도 이해할 수 있는 핵심 개념

2-1. 로드밸런서는 식당의 안내 직원입니다

How many people in your party? 何名様ですか?｜パシフィックブルー英会話

note.com

손님이 식당에 들어오면 안내 직원은 어느 테이블로 안내할지 결정합니다. 한 테이블에 손님이 몰리지 않도록 나눠 주고, 사용할 수 없는 테이블은 제외할 수도 있습니다.

로드밸런서도 같은 일을 합니다.

사용자

서비스 주소 하나로 요청

로드밸런서(Nginx)

요청을 처리할 서버 선택

서버 A

서버 B

서버 C

사용자는 서버 A, B, C의 주소를 직접 알 필요가 없습니다. 로드밸런서가 요청을 전달하고 응답을 돌려주기 때문입니다.

2-2. 서버를 고르는 네 가지 규칙

규칙	쉬운 설명	적합한 상황
라운드 로빈(Round Robin)	A, B, C 순서로 돌아가며 요청	서버 성능과 요청 특성이 비슷할 때
최소 연결(Least Connections)	현재 처리 중인 연결이 적은 서버 선택	요청마다 처리 시간이 크게 다를 때
가중치(Weighted)	정해 둔 비율에 따라 분배	서버 성능이 다르거나 새 버전을 일부 시험할 때
IP 해시(IP Hash)	같은 IP의 사용자를 같은 서버로 유도	IP를 기준으로 서버 고정이 필요한 경우

예를 들어 AI 에이전트의 요청은 간단한 질문이면 몇 초 만에 끝나지만, 긴 문서를 분석하거나 여러 도구를 호출하면 수십 초가 걸릴 수도 있습니다. 이런 경우에는 단순히 요청을 번갈아 보내는 것보다 현재 처리 중인 연결 수를 고려하는 방법이 도움이 될 수 있습니다.

다만 최소 연결이 항상 실제 작업량을 가장 정확하게 측정하는 것은 아닙니다. 연결 수가 적더라도 각 요청의 연산 비용이 매우 클 수 있기 때문입니다.

세션 고정(Sticky Session)의 주의점

IP 해시처럼 특정 사용자를 같은 서버에 연결하는 방식을 세션 고정이라고 합니다.

하지만 서버 A의 메모리에만 대화 기록이 저장되어 있다면, 사용자가 계속 서버 A로 연결되더라도 A가 재시작되거나 교체되면 대화 기록이 사라질 수 있습니다.

따라서 장기적으로는 사용자의 대화와 로그인 상태를 외부 저장소에 두어 서버가 바뀌어도 유지되도록 만드는 것이 좋습니다.

3. 헬스체크: 고장 난 서버를 어떻게 알아낼까요?

로드밸런서는 요청을 전달할 서버가 정상인지 확인해야 합니다.

Health Check Endpoints: /health, /livez, /readyz Guide | Webalert Blog

능동 헬스체크(Active Health Check)

로드밸런서가 주기적으로 서버에 먼저 상태를 물어봅니다. 정상 응답이 오지 않으면 서버를 요청 대상에서 제외합니다.

예: 일정 간격으로 `/health` 주소를 확인

Server Monitoring Basics: Uptime, Reachability, and Response Checks | Webalert Blog

수동 헬스체크(Passive Health Check)

별도의 확인 요청을 보내는 대신, 실제 사용자 요청이 실패한 기록을 보고 서버 상태를 판단합니다.

예: 서버 연결에 실패한 뒤 일정 시간 요청 대상에서 제외

강의에서 사용하는 무료 Nginx의 기본 HTTP 로드밸런싱 기능은 수동 헬스체크 방식입니다. 따라서 서버의 전원이 갑자기 꺼졌더라도 그 사실을 미리 확인해 놓은 것은 아닙니다.

여기서 중요한 설정이 세 가지 있습니다.

설정	의미
`max_fails`	서버를 실패한 것으로 판단하기까지 허용하는 실패 횟수
`fail_timeout`	실패를 판단하는 시간 구간이자, 서버를 일시적으로 제외하는 시간
`proxy_connect_timeout`	뒤쪽 서버와 연결을 맺기까지 기다리는 시간 제한

예를 들어 연결 제한을 60초로 두었는데 서버가 응답하지 않는다면, 사용자는 실패가 확인될 때까지 오래 기다릴 수 있습니다. 연결 제한을 2초로 낮추면 연결 시도 단계에서 기다리는 시간을 줄일 수 있습니다.

다만 연결 제한을 줄이는 것만으로 모든 지연이 해결되지는 않습니다. 서버에 연결된 뒤 응답이 늦는 경우에는 별도의 읽기 타임아웃 등의 설정도 검토해야 합니다.

4. 드레이닝: 서버를 끄기 전에 하던 일을 마치게 합니다

1. 서버를 요청 대상에서 제외합니다.

   새 요청이 해당 서버로 들어오지 않게 합니다.

2. 진행 중인 요청을 마무리합니다.

   사용자의 응답을 보내고 작업을 끝낼 시간을 줍니다.

3. 서버를 종료하고 새 버전으로 교체합니다.

   프로그램이나 컨테이너를 업데이트합니다.

4. 정상 작동을 확인한 뒤 다시 투입합니다.

   헬스체크를 통과한 서버에 새 요청을 보냅니다.

이 과정을 드레이닝(Draining)이라고 합니다.

예를 들어 AI가 긴 보고서를 작성하는 도중 서버를 강제로 종료하면 사용자는 응답을 받지 못할 수 있습니다. 드레이닝은 이런 상황을 줄이기 위해 새 요청은 받지 않으면서 진행 중인 요청을 끝낼 시간을 주는 방식입니다.

물론 요청이 무한정 길어지지 않도록 적절한 제한 시간과 종료 정책도 필요합니다.

5. Nginx는 웹 서버이자 리버스 프록시이자 로드밸런서입니다

Nginx는 하나의 프로그램으로 여러 역할을 수행할 수 있습니다.

역할	하는 일
웹 서버	HTML, 이미지, CSS 같은 정적 파일을 제공합니다.
리버스 프록시(Reverse Proxy)	사용자 요청을 받아 뒤쪽 애플리케이션 서버에 전달합니다.
로드밸런서(Load Balancer)	뒤쪽 서버가 여러 대라면 요청을 여러 서버로 분배합니다.

포워드 프록시와 리버스 프록시의 차이도 알아두면 좋습니다.

- 포워드 프록시: 사용자 쪽을 대신해 외부 서버에 요청합니다.
- 리버스 프록시: 서버 쪽을 대신해 외부 사용자의 요청을 받습니다.

Nginx는 이벤트 기반 비동기 처리 방식을 활용해 많은 연결을 효율적으로 관리합니다. 모든 연결에 별도의 처리 스레드를 하나씩 붙이는 방식과 달리, 적은 수의 작업 프로세스가 다수의 연결을 처리할 수 있습니다.

다만 작업 프로세스마다 일부 상태가 별도로 관리될 수 있기 때문에, 요청 분배 순서나 실패 기록을 여러 프로세스에서 공유해야 한다면 `upstream`의 `zone` 설정이 도움이 됩니다.

6. Nginx 설정 파일은 어떻게 읽을까요?

다음은 강의에 나온 설정을 단순화한 예시입니다.

```
upstream my_app {
    zone my_app 64k;
    server 10.0.0.1:8080 weight=3;
    server 10.0.0.2:8080;
    server 10.0.0.3:8080 backup;
}

server {
    listen 80;

    location / {
        proxy_pass http://my_app;
        proxy_connect_timeout 2s;
    }
}
```

코드를 처음 접하시는 입장에서는 세 부분만 구분하시면 됩니다.

① `upstream`

요청을 전달할 서버 목록과 각 서버의 분배 조건을 정합니다.

② `server`와 `location`

어느 포트로 들어온 요청을 받고, 어떤 경로의 요청을 처리할지 정합니다. 여기서 `server`는 Nginx의 수신 설정을 의미합니다.

③ `proxy_pass`

실제 요청을 `upstream`에 정의한 서버 목록으로 전달합니다.

설정의 주요 항목을 해석하면 다음과 같습니다.

- `weight=3`: 첫 번째 서버에 다른 기본 가중치 1인 서버보다 높은 비중을 줍니다.
- `backup`: 다른 일반 서버를 사용할 수 없을 때 대체 서버로 활용합니다.
- `zone my_app 64k`: 작업 프로세스 사이에서 업스트림 상태를 공유하도록 메모리 영역을 설정합니다.

- `proxy_connect_timeout 2s`: 뒤쪽 서버에 연결할 때 기다리는 시간을 2초로 제한합니다.

주의할 점은 `weight=3`과 기본 가중치 1이 있다고 해서 모든 짧은 실험에서 정확히 3 대 1로 관찰되는 것은 아니라는 것입니다. 연결 재사용, 작업 프로세스, 요청 수와 분배 방식에 따라 관찰 결과가 달라질 수 있습니다.

7. `reload`는 왜 중요할까요?

Nginx 설정을 변경했다고 해서 서비스를 완전히 종료하고 다시 시작할 필요는 없습니다.

일반적인 작업 순서는 다음과 같습니다.

설정 파일 수정

`nginx -t`

설정 문법 검사

`nginx -s reload`

새 설정을 적용

이전 작업 프로세스

기존 연결을 마무리

새 작업 프로세스

새 설정으로 요청 처리

`reload`는 Nginx를 통째로 중단했다가 다시 시작하는 대신, 새 설정을 읽고 새 작업 프로세스로 전환하는 방식입니다. 기존 작업 프로세스는 진행 중인 요청을 마무리하도록 종료 절차를 밟습니다.

하지만 Nginx의 reload가 성공했다고 해서 뒤쪽 애플리케이션 서버까지 자동으로 안전하게 교체되는 것은 아닙니다. 기존 서버를 너무 일찍 종료하면 진행 중인 요청이 끊길 수 있으므로 드레이닝이 필요합니다.

또한 서버 이름을 설정에 사용했다면, 새 컨테이너가 다른 IP 주소를 받았을 때 Nginx가 새로운 주소를 반영하도록 설정을 다시 읽는 과정이 필요할 수 있습니다.

8. 객관식 퀴즈 정답과 해설

퀴즈 1. 무료 Nginx 뒤의 서버 한 대가 갑자기 꺼졌다면?

정답: 2번

무료 Nginx의 기본 HTTP 로드밸런싱 기능은 능동 헬스체크를 수행하지 않습니다. 따라서 꺼진 서버로 요청이 전달되면 사용자는 연결을 기다리거나 실패를 경험할 수 있습니다. 실패

가 감지된 뒤에는 해당 서버가 일시적으로 제외되고 다른 서버로 요청이 전달될 수 있습니다.

퀴즈 2. `upstream`에 blue와 green을 함께 설정하고 reload하면?

정답: 2번. blue와 green이 번갈아 응답한다.

두 서버가 모두 정상이고 별도의 가중치나 백업 설정이 없다면 기본 라운드 로빈 방식으로 요청을 분배합니다.

단, 실제 관찰에서는 요청 수가 적거나 연결 재사용, 작업 프로세스별 상태 등의 영향을 받을 수 있으므로 반드시 엄격하게 번갈아 응답한다고 단정하기보다는 분배 규칙이 적용된다고 이해하는 것이 정확합니다.

9. 실습에서 확인해야 할 체크리스트

- [ ] Nginx를 통해 요청이 서버 여러 대로 전달되는지 확인합니다.
- [ ] 라운드 로빈, 최소 연결, 가중치, IP 해시의 차이를 확인합니다.
- [ ] 서버 하나를 중단하고 실패가 발생하는 시점과 이후 동작을 관찰합니다.
- [ ] `proxy_connect_timeout` 값을 바꾸어 연결 실패까지 기다리는 시간이 어떻게 달라지는지 측정합니다.
- [ ] 서버를 드레이닝한 뒤 교체하는 순서를 확인합니다.
- [ ] `nginx t`로 설정을 검사하고 `nginx s reload`로 적용합니다.
- [ ] reload 직후 이전 서버를 바로 종료했을 때와 요청을 마무리한 뒤 종료했을 때를 비교합니다.

10. 반드시 기억해야 할 내용

1. 로드밸런서는 사용자 요청을 적절한 서버로 전달하는 입구입니다.
2. 라운드 로빈, 최소 연결, 가중치, IP 해시는 각각 다른 기준으로 서버를 선택합니다.
3. 무료 Nginx의 기본 HTTP 로드밸런싱은 수동 헬스체크를 사용하므로 첫 실패 요청을 완전히 없애지는 못합니다.
4. 드레이닝은 새 요청을 차단하면서 기존 요청을 마무리하도록 하는 과정입니다.
5. Nginx는 웹 서버, 리버스 프록시, 로드밸런서 역할을 수행할 수 있습니다.
6. `nginx t`는 설정 검사, `nginx s reload`는 새 설정 적용을 위한 명령입니다.

7. 로드밸런서의 설정 변경과 애플리케이션 서버의 교체는 별개입니다. 실제 무중단 운영을 위해서는 둘을 함께 고려해야 합니다.

11. 한 문장 요약

로드밸런서는 요청을 적절한 서버에 분배하고 장애 서버를 제외하며, Nginx의 설정 변경과 드레이닝을 함께 활용하면 서비스를 중단하지 않고 서버를 교체하는 데 도움이 됩니다.

Chapter04

/강의정리 챕터 04. 시작 상태 맞추기

1. 이번 실습의 목적

이번 실습의 목적은 내 컴퓨터에서 블루-그린(Blue-Green) 무중단 배포 환경을 직접 만들어 보는 것입니다.

앞선 챕터에서 배운 로드밸런서와 블루-그린 배포를 실제 Docker 컨테이너로 구성합니다. Google Cloud는 사용하지 않습니다.

핵심은 서비스를 두 벌 준비하고, Nginx 설정 한 줄을 바꿔 사용자 요청이 어느 서비스로 갈지 전환하는 것입니다.

Why Docker is a Game-Changer for Development, Not Just Production - DEV Community

Docker 컨테이너 2개: Blue와 Green

같은 에이전트 이미지를 사용하지만, 화면에 표시되는 배너와 버전 정보는 다르게 설정합니다.

Deploy Your First NGINX Web Server on AWS EC2 | by Kajal | Medium

Nginx 컨테이너 1개: 교통정리 담당자

사용자의 요청을 받아 현재 활성화된 Blue 또는 Green으로 전달합니다.

2. 중학생도 이해할 수 있는 비유

학교 급식실에 식사 배식대가 두 개 있다고 생각해 보겠습니다.

- Blue 배식대: 현재 학생들이 이용하는 배식대
- Green 배식대: 새로 준비해 놓은 배식대

- Nginx: 학생들을 어느 배식대로 보낼지 결정하는 안내 직원

새 배식대를 준비하는 동안 기존 배식대를 계속 이용할 수 있습니다. 새 배식대의 준비가 끝나면 안내 직원의 지시만 바꾸면 됩니다.

이번 실습에서는 안내 직원의 역할을 하는 Nginx가 다음 설정으로 목적지를 정합니다.

```
server blue:8080;
```

나중에 이 줄을 다음처럼 바꾸면 됩니다.

```
server green:8080;
```

즉, 서비스를 새로 만드는 작업과 사용자 요청의 목적지를 바꾸는 작업을 분리하는 것이 핵심입니다.

단, 이번 챕터의 시작 단계에서는 아직 Green으로 전환하지 않습니다. 먼저 Blue가 정상적으로 응답하는 상태를 만듭니다.

3. 실습을 시작하기 전에 확인할 것

실습 환경을 처음부터 전부 만들기 전에, 이미 준비된 것이 있는지 확인합니다. 이 단계에서는 확인만 하고, 파일이나 환경을 변경하지 않습니다.

확인 대상	실행 명령	정상 상태
Python 가상환경과 `httpx`	`python -c "import httpx"`	오류 없이 종료
Docker 실행 상태	`docker info`	Docker 정보가 출력됨
Docker 설정 파일	`ls Dockerfile .dockerignore`	두 파일이 존재함
에이전트 이미지	`docker images friend-agent:local`	이미지가 목록에 표시됨

Windows PowerShell에서 가상환경을 켤 때는 다음 명령을 사용합니다.

```
.\.venv\Scripts\Activate.ps1
```

이미지가 없다면 저장소 루트에서 다음 명령으로 만듭니다.

```
docker build -t friend-agent:local .
```

여기서 `friend-agent:local`은 로컬에서 사용할 이미지 이름입니다. 이미지가 없으면 컨테이너를 실행할 수 없으므로 먼저 준비해야 합니다.

주의: 기존 실습을 이어서 진행한다면 이미 있는 네트워크나 컨테이너를 무조건 다시 만들거나 삭제하지 말고, 먼저 현재 상태를 확인해야 합니다.

4. 이번에 만들 구조

내 컴퓨터

브라우저 · 요청 확인 스크립트

localhost:8080

8080 → 80

Nginx 컨테이너 `bg-nginx`

요청 전달 · upstream 설정으로 목적지 결정

현재 활성 경로

전환 후 경로

Blue

blue:8080

현재 활성

Green

green:8080

대기 중

세 컨테이너는 모두 Docker 네트워크 bg-lab에 연결됩니다. Blue와 Green은 호스트 컴퓨터에 포트를 직접 공개하지 않습니다.

이 구조에서 꼭 이해해야 할 점이 세 가지 있습니다.

1. 컨테이너 이름으로 통신합니다. 같은 Docker 네트워크에 있는 Nginx는 `blue:8080` 또는 `green:8080`으로 요청을 전달할 수 있습니다.
2. 외부 요청은 Nginx만 받습니다. 내 컴퓨터의 8080번 포트가 Nginx의 80번 포트와 연결됩니다.
3. 설정 파일을 호스트와 연결합니다. 내 컴퓨터에서 설정을 수정하면 컨테이너가 그 변경된 파일을 읽을 수 있도록 폴더 전체를 연결합니다.

5. 환경변수 두 개의 역할

Blue와 Green은 동일한 이미지로 실행하지만 환경변수 값을 다르게 줍니다.

환경변수	Blue	Green	의미
`BANNER`	`BLUE`	`GREEN`	화면에 표시할 배너
`K_REVISION`	`blue`	`green`	`/version` 응답에서 구분할 버전 정보
`LLM_PROVIDER`	`fake`	`fake`	실제 LLM 대신 가짜 LLM 사용

`K_REVISION`은 원래 Cloud Run 리비전 이름과 관련된 환경변수이지만, 이번 실습에서는 로컬 컨테이너를 구별하기 위해 직접 지정합니다.

또한 `LLM_PROVIDER=fake`를 사용하므로 실제 LLM API 호출이나 API 키가 필요하지 않습니다. 배포 전환이 제대로 되는지만 확인하면 되기 때문입니다.

6. 실습 순서

1. 1단계. Nginx 설정 파일 만들기

   저장소 밖에 `bluegreen-lab` 폴더를 만들고, 그 안에 `default.conf`를 둡니다. 최초 설정은 Blue만 가리킵니다.

   ```
   upstream active_app {
       zone active_app 64k;
       server blue:8080;
   }

   server {
       listen 80;
       location / {
           proxy_pass http://active_app;
       }
   }
   ```

   `upstream active_app`은 요청을 보낼 대상 그룹이고, `proxy_pass`는 해당 그룹으로 요청을 전달하라는 의미입니다. `zone`은 여러 Nginx 작업 프로세스가 upstream 상태를 공유할 수 있도록 하는 설정입니다.

2. 2단계. Docker 네트워크 만들기

   ```
   docker network create bg-lab
   ```

   같은 네트워크에 연결된 컨테이너끼리 이름으로 통신할 수 있도록 합니다.

3. 3단계. Blue와 Green 실행하기

```
docker run -d --name blue --network bg-lab -e LLM_PROVIDER=fake -e K_REVISION=blue
-e BANNER=BLUE friend-agent:local

docker run -d --name green --network bg-lab -e LLM_PROVIDER=fake -e K_REVISION=gre
en -e BANNER=GREEN friend-agent:local
```

- `d`는 백그라운드 실행, `-name`은 컨테이너 이름 지정, `e`는 환경변수 설정입니다. `p` 옵션이 없으므로 Blue와 Green의 서비스 포트는 호스트에 직접 공개되지 않습니다.

4. 4단계. Nginx 실행하기

```
docker run -d --name bg-nginx --network bg-lab -p 8080:80 -v "$HOME/bluegreen-la
b:/etc/nginx/conf.d" nginx:1.27-alpine
```

- `p 8080:80`은 내 컴퓨터의 8080번 포트를 컨테이너의 80번 포트에 연결합니다. `v`는 호스트의 설정 폴더를 컨테이너의 Nginx 설정 폴더에 연결합니다.

5. 5단계. 실제 설정과 응답 확인하기

```
docker ps
docker exec bg-nginx cat /etc/nginx/conf.d/default.conf
```

이어서 브라우저에서 다음 두 주소를 확인합니다.

- `http://localhost:8080` — BLUE 배너 표시
- `http://localhost:8080/version` — `revision` 값이 `blue`

`docker ps`에서는 세 컨테이너가 실행 중인지, 호스트 포트 매핑은 Nginx에만 있는지 확인합니다.

7. Windows PowerShell에서 주의할 점

Windows에서도 Docker 명령은 대부분 동일하게 사용할 수 있습니다. 다만 파일을 만들거나 수정하는 명령은 차이가 있습니다.

macOS/Linux	Windows PowerShell
`mkdir -p ~/bluegreen-lab`	`mkdir $HOME\bluegreen-lab`
`cat > ... <<'EOF'`	편집기에서 파일을 만들고 내용 붙여넣기
`sed -i ...`	편집기에서 해당 설정 줄 직접 수정

macOS/Linux	Windows PowerShell
`head -5`	`Select-Object -First 5`

특히 폴더 연결에 `~`를 따옴표 안에 넣으면 경로가 의도대로 확장되지 않을 수 있습니다. 교안처럼 `$HOME`을 사용하는 편이 안전합니다.

설정 파일을 수정한 후에는 다음 명령으로 문법을 확인하고 적용합니다.

```
docker exec bg-nginx nginx -t
docker exec bg-nginx nginx -s reload
```

첫 번째 명령은 설정 검사, 두 번째 명령은 설정 다시 읽기입니다. 설정 파일이 올바르더라도 서비스 컨테이너가 준비되지 않았거나 네트워크 연결이 잘못되어 있으면 요청은 실패할 수 있으므로, 설정 검사와 실제 요청 확인은 별개입니다.

8. AI 개발자와 1인 창업가가 알아야 하는 이유

이 실습은 단순히 Docker 명령을 익히는 과정이 아닙니다. AI 서비스를 사용자에게 제공할 때 장애 위험을 줄이는 배포 구조를 이해하는 과정입니다.

예를 들어 준비 중인 MSM(My-Story-Maker)에 적용한다면 다음과 같이 생각할 수 있습니다.

- 기존 버전은 Blue에서 사용자 요청을 처리합니다.
- 새로운 기능을 포함한 버전은 Green에 별도로 준비합니다.
- Green이 정상적으로 실행되는지 확인합니다.
- Nginx의 목적지를 Green으로 변경합니다.
- 문제가 생기면 Blue로 다시 전환할 수 있습니다.

다만 이 실습만으로 완전한 무중단 배포가 보장되는 것은 아닙니다. 실제 운영에서는 헬스체크, 기존 요청의 종료 처리, 데이터베이스 호환성, 롤백 절차까지 함께 설계해야 합니다.

9. 꼭 기억할 내용

- 이미지(Image): 컨테이너를 만들 때 사용하는 실행 패키지입니다.
- 컨테이너(Container): 이미지를 실제로 실행한 인스턴스입니다.
- Docker 네트워크: 컨테이너끼리 이름으로 통신하게 해 줍니다.
- Nginx: 사용자 요청을 현재 활성화된 서비스로 전달합니다.
- `p`: 호스트와 컨테이너의 포트를 연결합니다.

- `v`: 호스트 파일이나 폴더를 컨테이너에 연결합니다.
- `nginx t`: 설정이 올바른지 검사합니다.
- `nginx s reload`: 실행 중인 Nginx가 설정을 다시 읽게 합니다.

10. 한 문장 요약

이번 챕터는 Docker로 Blue와 Green 서비스, 그리고 Nginx를 구성하고, Nginx 설정을 통해 사용자 요청의 목적지를 바꿀 수 있는 무중단 배포 실습 환경을 준비하는 과정입니다.

Chapter05

`/강의정리` 챕터 05. 요청 확인 스크립트

1. 이번 강의의 목적

이번 강의에서는 앞서 만든 Docker 기반 블루-그린 배포 환경에서 실제로 요청을 계속 보내면서 배포 전환 중 오류가 발생하는지 측정합니다.

챕터 04에서는 Blue, Green, Nginx를 준비했습니다. 이번에는 사용자가 계속 서비스를 이용하는 상황에서 다음 네 가지를 실험합니다.

1. Blue에서 Green으로 전환하고 다시 롤백하기
2. Blue와 Green에 3:1 비율로 요청 보내기
3. Green 서버가 갑자기 멈췄을 때 오류 측정하기
4. 서버를 전환하거나 재시작할 때 대화와 약속 데이터가 어떻게 되는지 확인하기

가장 중요한 결론은 다음과 같습니다.

요청이 끊기지 않는 것과 사용자가 하던 일을 계속할 수 있는 것은 서로 다른 문제입니다.

2. 중학생도 이해할 수 있는 비유

두 개의 고객 상담실이 있다고 생각해 보겠습니다.

- Blue 상담실: 기존 상담원이 근무하는 곳
- Green 상담실: 새로운 상담원이 근무하는 곳
- Nginx: 고객을 어느 상담실로 보낼지 결정하는 안내 직원
- 요청 확인 스크립트: 고객이 상담을 요청할 때마다 성공 여부를 기록하는 관찰자

새 상담실로 안내를 바꿨는데 고객이 기다리지 않고 상담을 받았다면 요청은 성공한 것입니다.

그런데 새 상담원이 고객의 이전 상담 내용을 모른다면 어떨까요? 고객은 연결은 되었지만 같은 설명을 다시 해야 합니다.

또 기존 상담실을 닫았는데 안내 직원이 그 사실을 모른다면, 고객은 문이 닫힌 상담실 앞에서 기다리다가 실패할 수 있습니다.

이것이 이번 실습에서 구분해야 하는 두 문제입니다.

- 가용성 문제: 요청에 정상적으로 응답했는가?
- 상태 유지 문제: 사용자의 대화와 작업 내용이 이어지는가?

3. 요청 확인 스크립트는 무엇을 측정할까요?

실습 파일은 `scripts/watch_requests.py`입니다.

```
python scripts/watch_requests.py http://localhost:8080
```

이 스크립트는 약 0.2초마다 `/version`에 요청을 보내고, 응답 코드와 응답한 revision을 기록합니다.

요청 주기

약 0.2초마다 1회

초당 약 5회 요청

응답 제한 시간

2초

연결 실패 또는 제한 시간 초과도 실패로 집계

최종 결과

전체 요청 수 · 실패 수 · Blue 응답 수 · Green 응답 수

측정 시간을 지정하려면 다음처럼 실행합니다.

```
python scripts/watch_requests.py http://localhost:8080 --seconds 30
```

최대 5분까지 측정할 수 있으며, `Ctrl+C`로 중단할 수도 있습니다.

왜 브라우저 새로고침만으로는 부족할까요?

브라우저를 몇 번 새로고침하는 것은 제한된 순간만 확인하는 방법입니다. 전환하는 찰나에 요청이 실패했는지, 서버가 응답을 늦게 했는지, 실제로 어느 버전이 응답했는지를 정확하게 파악하기 어렵습니다.

요청 확인 스크립트를 사용하면 배포 전후의 요청을 연속적으로 기록해 전환 시점과 오류 발생 시점을 비교할 수 있습니다.

4. 실험 ① Blue에서 Green으로 전환하고 롤백하기

4-1. Green이 정상인지 먼저 확인합니다

Green은 호스트 컴퓨터에 포트를 직접 공개하지 않았으므로 Nginx 컨테이너 안에서 확인합니다.

```
docker exec bg-nginx wget -qO- http://green:8080/health
docker exec bg-nginx wget -qO- http://green:8080/version
```

두 주소가 정상적으로 응답하면 Green으로 전환할 준비가 된 것입니다.

4-2. 설정 한 줄을 변경합니다

기존 설정은 다음과 같습니다.

```
server blue:8080;
```

이를 다음과 같이 바꿉니다.

```
server green:8080;
```

macOS에서 사용하는 명령은 다음과 같습니다.

```
sed -i '' 's/server blue:8080;/server green:8080;/' ~/bluegreen-lab/default.conf
docker exec bg-nginx nginx -t
docker exec bg-nginx nginx -s reload
```

Windows에서는 편집기로 설정 파일을 수정한 후 검사와 reload 명령을 실행합니다.

순서는 반드시 `수정 → 검사 → reload` 입니다. 설정 검사가 실패하면 reload하지 않아야 합니다.

4-3. 어떤 결과가 나와야 할까요?

- 브라우저의 배너가 BLUE에서 GREEN으로 바뀝니다.
- `/version`의 `revision` 값이 `green`으로 바뀝니다.
- 요청 확인 스크립트에는 전환 전 Blue 응답과 전환 후 Green 응답이 기록됩니다.
- 정상적으로 전환되었다면 실패가 0건일 수 있습니다.

Nginx는 설정을 다시 읽을 때 기존 작업 프로세스를 즉시 강제 종료하지 않고, 기존 요청을 처리하도록 한 뒤 새 설정을 적용합니다. 따라서 정상적인 조건에서는 서비스 전체를 재시작하지 않고 전환할 수 있습니다.

다만 실패 0건은 이번 측정 구간에서 실패가 관찰되지 않았다는 뜻이지, 모든 상황에서 무중단이 보장된다는 뜻은 아닙니다.

4-4. 롤백은 왜 빠를까요?

Green에 문제가 생겼다고 가정하고 설정을 다시 Blue로 돌립니다.

```
sed -i '' 's/server green:8080;/server blue:8080;/' ~/bluegreen-lab/default.conf
docker exec bg-nginx nginx -t
docker exec bg-nginx nginx -s reload
```

Blue 컨테이너를 계속 실행 중인 상태로 유지했기 때문에 이전 버전을 다시 빌드하거나 실행할 필요 없이 목적지만 바꾸면 됩니다.

이것이 블루-그린 배포의 중요한 장점입니다. 이전 버전을 즉시 없애지 않고 유지하면 문제가 생겼을 때 빠르게 돌아갈 수 있습니다.

5. 실험 ② Blue와 Green에 3:1로 요청 보내기

이번에는 한쪽으로만 보내지 않고 두 버전에 요청을 나눠 보냅니다.

```
upstream active_app {
    zone active_app 64k;
    server blue:8080 weight=3;
    server green:8080;
}
```

여기서 `weight=3`은 Blue의 가중치가 Green보다 세 배라는 뜻입니다.

예상되는 요청 분배

Blue

약 75%

Green

약 25%

예상 비율입니다. 측정 시간, 요청 수, 서버 상태 등에 따라 실제 결과는 달라질 수 있습니다.

설정 파일을 위 내용으로 바꾼 후 검사하고 적용합니다.

```
docker exec bg-nginx nginx -t
docker exec bg-nginx nginx -s reload
python scripts/watch_requests.py http://localhost:8080 --seconds 30
```

요약 결과에서 Blue가 약 75%, Green이 약 25%의 요청에 응답하는지 확인합니다.

이 방식은 새 버전을 모든 사용자에게 공개하기 전에 일부 요청만 보내 보는 카나리 배포의 기초가 됩니다.

여기서 발견할 수 있는 또 다른 문제

두 버전에 요청을 나눠 보냈을 때 화면의 배너가 번갈아 바뀌는 것 자체는 정상입니다.

하지만 서비스가 대화형 AI라면 문제가 생길 수 있습니다. Blue가 기억하는 대화와 Green이 기억하는 대화가 서로 다르다면, 사용자의 다음 요청이 어느 서버로 가느냐에 따라 이전 대화를 기억하기도 하고 기억하지 못하기도 합니다.

즉, 요청을 나누는 기술만으로는 대화 상태가 공유되지 않습니다.

6. 실험 ③ Green이 갑자기 멈추면?

이번에는 두 서버에 요청을 나누는 상태에서 Green을 강제로 멈춥니다.

첫 번째 터미널에서 60초 동안 요청을 측정합니다.

```
python scripts/watch_requests.py http://localhost:8080 --seconds 60
```

두 번째 터미널에서는 약 10초 뒤에 다음 명령을 실행합니다.

```
docker stop green
```

6-1. 연결 대기 시간이 길 때

기본 연결 제한 시간이 60초인 설정에서는 Nginx가 Green에 연결하려다 오랫동안 기다릴 수 있습니다.

하지만 요청 확인 스크립트는 2초 안에 응답이 오지 않으면 실패로 집계합니다. 따라서 Nginx가 연결 실패를 확정하기 전에 스크립트가 먼저 시간 초과로 판단할 수 있습니다.

이때 나타나는 현상은 다음과 같습니다.

- Green이 멈춘 직후부터 일부 요청이 실패합니다.
- 실패 수가 증가합니다.
- Nginx가 Green을 제외한 뒤에는 Blue가 응답하는 요청이 이어집니다.

여기서 알아야 할 점은 Nginx가 서버의 고장을 알아차리는 시간과 사용자가 기다리는 시간은 다르다는 것입니다.

6-2. 연결 대기 시간을 1초로 줄이면?

먼저 Green을 다시 실행합니다.

```
docker start green
```

설정 파일을 다음처럼 수정합니다.

```
upstream active_app {
    zone active_app 64k;
    server blue:8080 weight=3;
    server green:8080;
}

server {
    listen 80;
    location / {
        proxy_pass http://active_app;
        proxy_connect_timeout 1s;
    }
}
```

수정한 후 설정을 검사하고 적용합니다.

```
docker exec bg-nginx nginx -t
docker exec bg-nginx nginx -s reload
```

이제 다시 60초 측정을 시작하고, 약 10초 뒤 Green을 멈춥니다.

```
python scripts/watch_requests.py http://localhost:8080 --seconds 60
```

```
docker stop green
```

이번에는 Green에 연결을 시도하다 실패하는 데 걸리는 시간을 1초로 줄였기 때문에, 다른 서버로 재시도하는 요청이 스크립트의 2초 제한 안에 성공할 가능성이 높아집니다.

다만 실패가 반드시 0건이 되는 것은 아닙니다. 재시도 동작, 응답 시간, 연결 상태 등에 따라 실패가 남을 수 있습니다. 또한 `proxy_connect_timeout`은 연결 시도에 적용되는 제한 시간이지, 전체 요청 처리 시간의 제한이 아닙니다.

실제 서비스에서는 무조건 제한 시간을 짧게 설정하기보다 정상적인 연결 시간과 서버 부하를 측정해 적절하게 정해야 합니다.

7. 실험 ④ 요청은 성공했는데 대화는 왜 사라질까요?

이번 강의에서 가장 중요한 실험입니다.

다음 순서로 확인합니다.

1. Blue가 활성화된 상태에서 대화를 만들고 약속이 화면에 표시되는지 확인합니다.
2. Green으로 전환하고 새로고침합니다.
3. 대화와 약속이 보이는지 확인합니다.
4. 다시 Blue로 롤백하고 새로고침합니다.
5. Blue 컨테이너를 재시작한 뒤 대화와 약속이 남아 있는지 확인합니다.

결과를 어떻게 이해해야 할까요?

요청은 정상적으로 전환될 수 있습니다.

Nginx가 Blue에서 Green으로 요청을 보내는 데 성공하면, 사용자는 새 서버의 응답을 받을 수 있습니다.

하지만 대화는 서버 메모리에 남아 있을 수 있습니다.

Blue 메모리에만 대화가 저장되어 있다면 Green은 그 내용을 알 수 없습니다. Blue로 돌아오면 대화가 다시 보일 수 있습니다.

컨테이너를 재시작하면 메모리의 데이터가 사라질 수 있습니다.

데이터가 외부 저장소에 저장되지 않았다면 컨테이너 재시작 후에는 대화와 약속을 복구하지 못할 수 있습니다.

이것은 앞서 배운 무상태(Stateless) 설계와 연결됩니다.

서버 메모리에만 사용자 상태를 저장하면 요청이 다른 서버로 이동할 때 문제가 생깁니다. 대화와 약속 같은 지속되어야 하는 데이터는 데이터베이스 등 외부의 공용 저장소에 저장하고, 어느 서비스 인스턴스에서 요청을 처리하더라도 같은 정보를 읽을 수 있도록 설계해야 합니다.

8. 1인 AI 창업에 적용한다면

AI 상담 서비스나 MSM처럼 사용자의 글과 대화, 기억을 다루는 서비스에서는 배포 중 요청이 성공하는 것만 확인해서는 충분하지 않습니다.

점검 영역	확인할 질문
요청 가용성	배포 전환 중 오류가 발생했나요?
응답 지연	사용자가 얼마나 오래 기다렸나요?
상태 연속성	이전 대화와 약속이 새 버전에서도 보이나요?
데이터 지속성	컨테이너를 재시작해도 중요한 데이터가 남아 있나요?
롤백	새 버전에 문제가 생기면 이전 버전으로 안전하게 돌아갈 수 있나요?

운영 환경에서는 이 항목들을 별도로 검증해야 합니다. 특히 대화 데이터의 저장 방식이 잘못되어 있다면 요청 오류가 0건이더라도 사용자 경험은 나빠질 수 있습니다.

9. 주관식 퀴즈 모범 답안

질문: 블루그린 전환 뒤 실패는 0건이었지만 대화가 사라졌고, Green을 멈춘 실험에서는 요청이 실패했습니다. 두 결과를 나란히 놓고, 무중단 배포를 확인할 때 '요청이 끊기지 않는 것'과 '사용자가 하던 일이 이어지는 것'을 왜 따로 봐야 할까요?

모범 답안

요청이 끊기지 않는다는 것은 서버를 전환하는 동안에도 사용자의 요청에 정상적으로 응답할 수 있다는 의미입니다. 반면 사용자가 하던 일이 이어진다는 것은 이전 대화와 약속 같은 상태 정보가 새 서버에서도 유지된다는 의미입니다.

Blue에서 Green으로 전환했을 때 요청은 성공했지만 대화가 사라졌다면, 네트워크와 요청 전달은 정상이어도 사용자 상태가 서버 메모리에만 저장되어 있다는 뜻일 수 있습니다. 반대로 Green이 멈췄을 때 요청이 실패했다면, 서버 장애를 감지하고 다른 서버로 전환하는 과정에서 가용성 문제가 발생한 것입니다.

따라서 무중단 배포를 제대로 확인하려면 요청 성공률과 응답 시간뿐 아니라 데이터의 지속성, 대화 상태의 공유, 장애 복구와 롤백까지 별도로 검증해야 합니다.

10. 꼭 기억할 내용

- 요청 확인 스크립트는 배포 전환 중 오류와 revision 변화를 기록합니다.
- Blue와 Green의 가중치를 3:1로 설정하면 대략 75% 대 25%로 요청을 분배합니다.
- 서버가 멈췄을 때 Nginx의 연결 대기 시간이 길면 사용자 요청도 실패할 수 있습니다.
- `proxy_connect_timeout`은 연결 시도 대기 시간을 조절하지만 모든 지연을 해결하지는 않습니다.
- Nginx 설정 검사와 reload를 거쳐 안전하게 전환해야 합니다.
- 서버 메모리에만 저장된 대화는 다른 서버에서 보이지 않을 수 있습니다.
- 무중단 배포는 요청 가용성과 사용자 상태 연속성을 모두 확인해야 합니다.

한 문장 요약

이번 챕터에서는 요청 확인 스크립트로 블루-그린 전환, 가중치 분배, 서버 장애의 영향을 측정하고, 진정한 무중단 서비스에는 요청 성공뿐 아니라 사용자 데이터와 대화 상태의 지속성도 필요하다는 점을 배웁니다.

Chapter06

/강의정리 챕터 06. 컨테이너 밖에 데이터 두기

1. 이번 강의의 목적

이번 강의의 핵심은 대화와 약속 데이터를 컨테이너 메모리가 아니라 외부 데이터베이스에 저장하는 것입니다.

지난 챕터에서는 Blue에서 Green으로 전환하면 대화가 사라지는 문제를 확인했습니다. Blue와 Green이 각각 자신의 메모리에만 대화 내용을 보관했기 때문입니다.

이번에는 관리형 데이터베이스인 Supabase를 연결해 이 문제를 해결합니다.

지난 챕터와 이번 챕터 비교

이전 구조: 컨테이너 안에 데이터 저장

Blue

대화 A

Green

대화 B

서로 다른 메모리를 사용하므로 전환하면 대화가 달라질 수 있습니다.

데이터를 외부 저장소로 이동

새 구조: 두 컨테이너가 같은 데이터베이스 사용

Blue

Green

Supabase 데이터베이스

대화 · 요약 · 기억 · 약속

어느 컨테이너가 요청을 처리하더라도 공용 데이터베이스에서 같은 사용자 데이터를 읽습니다.

2. 중학생도 이해할 수 있는 비유

Blue와 Green을 각각 다른 상담원이라고 생각해 보겠습니다.

이전에는 상담원마다 자기 수첩에 고객의 대화를 기록했습니다. 고객이 다른 상담원에게 가면 이전 상담 내용을 알 수 없었습니다.

이번에는 상담 내용을 모두가 함께 사용하는 중앙 고객 기록 시스템에 저장합니다.

- Blue 상담원은 중앙 기록을 읽고 상담합니다.
- Green 상담원도 같은 기록을 읽고 상담합니다.
- Blue가 퇴근하거나 재시작되어도 기록은 남습니다.
- 고객이 다른 상담원에게 연결되어도 이전 대화를 이어갈 수 있습니다.

이것이 외부 데이터베이스를 사용하는 이유입니다.

여기서 중요한 것은 컨테이너를 무상태(Stateless)로 만든다는 의미입니다. 서버가 사용자 대화의 유일한 사본을 메모리에 가지고 있지 않도록 하는 것입니다. 그렇다고 서비스에 상태가 없어지는 것은 아닙니다. 상태를 서버 메모리에서 외부 저장소로 옮기는 것입니다.

3. Supabase는 무엇인가요?

Supabase는 PostgreSQL 데이터베이스와 사용자 인증 기능 등을 제공하는 관리형 서비스입니다.

구성 요소	쉬운 설명	이번 실습에서 하는 일
PostgreSQL	데이터를 표 형태로 저장하는 데이터베이스	대화, 기억, 약속 저장

구성 요소	쉬운 설명	이번 실습에서 하는 일
Auth	사용자가 누구인지 확인하는 기능	게스트 사용자 식별
RLS	데이터 행마다 접근 권한을 제한하는 기능	사용자별 데이터 보호
관리형 서비스	서버 운영의 많은 부분을 서비스 제공자가 맡는 방식	직접 DB 서버를 설치·운영하는 부담 감소

왜 데이터베이스뿐 아니라 로그인 기능도 필요할까요?

여러 사용자의 대화가 하나의 데이터베이스에 저장되므로, 시스템은 어떤 사용자의 데이터인지 구별하고 해당 사용자에게 허용된 데이터만 제공해야 합니다.

이번 실습에서는 회원가입 절차 없이 사용할 수 있는 익명 로그인을 활용합니다. 브라우저마다 게스트 사용자 하나가 만들어지므로, 다른 브라우저에서는 다른 사용자의 데이터가 보이지 않는지 확인할 수 있습니다.

4. 실습 ① 시험용 Supabase 프로젝트 만들기

실습 데이터가 실제 서비스 데이터와 섞이지 않도록 별도의 시험용 프로젝트를 만듭니다.

1. Supabase 계정과 프로젝트 생성

   GitHub 계정으로 가입한 후 `friend-agent-staging`처럼 시험용임을 알 수 있는 이름으로 프로젝트를 만듭니다. 데이터베이스 비밀번호는 비밀번호 관리 도구에 안전하게 보관합니다.

2. 테이블 생성

   SQL Editor에서 저장소의 `supabase/migrations/0001_init.sql` 내용을 실행합니다. 캐릭터, 대화, 요약, 기억, 약속 등을 저장할 테이블이 만들어집니다.

3. RLS 활성화 확인

   모든 테이블에 행 단위 접근 제한(Row Level Security)이 켜져 있는지 확인합니다. RLS는 사용자별 데이터 접근을 제한하는 중요한 보안 장치입니다.

4. 익명 로그인 활성화

   `Authentication > Sign In / Providers`에서 `Allow anonymous sign-ins`를 켭니다. 익명 로그인 한도도 확인합니다. 여러 사용자가 같은 네트워크를 이용하면 IP 기반 한도에 걸릴 수 있습니다.

5. API 연결 정보 확인

   `Project Settings > API Keys`에서 프로젝트 URL, publishable key, secret key의 위치를 확인합니다. 키 값은 채팅에 붙여 넣거나 터미널에 출력하지 않습니다.

교안에는 2026년 10월 기준 무료 플랜의 활성 프로젝트 수와 일시정지 조건이 안내되어 있습니다. 이러한 무료 플랜 정책은 변경될 수 있으므로 실제 프로젝트를 만들 때는 현재 Supabase 대시보드의 안내를 기준으로 판단하시면 됩니다.

5. 세 가지 연결 정보와 보안

이번 실습에서 가장 주의해야 할 부분입니다.

값	용도	취급 원칙
`SUPABASE_URL`	어떤 Supabase 프로젝트에 연결할지 지정	앱 설정에 사용
`SUPABASE_PUBLISHABLE_KEY`	브라우저에서 인증 및 공개 API 접근에 사용	노출 자체보다 접근 권한과 RLS 설정이 중요
`SUPABASE_SECRET_KEY`	서버에서 높은 권한으로 데이터 접근	서버에만 보관하고 절대 브라우저에 노출하지 않음

Publishable key는 공개용 키이지만, 이것만으로 모든 데이터가 자동으로 안전해지는 것은 아닙니다. 테이블에 RLS가 올바르게 설정되어 있어야 하고, 인증된 사용자에게 허용된 데이터만 읽고 쓸 수 있도록 정책을 구성해야 합니다.

반면 secret key는 높은 권한으로 RLS를 우회할 수 있으므로 유출되면 심각한 문제가 발생할 수 있습니다.

`.env` 파일에 넣을 다섯 줄

저장소 루트의 `.env` 파일에는 다음 항목이 필요합니다.

```
STORAGE=supabase
AUTH=supabase
SUPABASE_URL=직접_입력
SUPABASE_PUBLISHABLE_KEY=직접_입력
SUPABASE_SECRET_KEY=직접_입력
```

위 내용은 형식을 설명하기 위한 예시입니다. 실제 값은 Supabase 대시보드에서 확인한 뒤 편집기로 직접 입력하시면 됩니다.

- `STORAGE=supabase`: 데이터 저장소로 Supabase 사용
- `AUTH=supabase`: 인증 기능으로 Supabase 사용
- 나머지 세 항목: 프로젝트 연결 및 인증에 필요한 값

`.env`는 Git에 커밋하지 않아야 합니다. 서버에 배포할 때는 secret key를 `.env` 파일에 넣어 배포하기보다는 Secret Manager 같은 비밀정보 저장소를 통해 관리하는 것이 적절합니

다.

6. 실습 ② Blue와 Green에 데이터베이스 연결하기

이제 기존 컨테이너를 제거하고, 같은 Supabase 프로젝트를 사용하도록 다시 실행합니다.

먼저 저장소 루트에서 실행합니다.

```
docker rm -f blue green
```

이 명령은 지정한 Blue와 Green 컨테이너를 제거합니다. 다른 컨테이너를 지우는 명령은 아니지만, 두 컨테이너 안에만 저장되어 있던 데이터가 있다면 함께 사라질 수 있으므로 실습 대상이 맞는지 확인해야 합니다.

그다음 `.env`의 설정을 적용해 두 컨테이너를 실행합니다.

```
docker run -d --name blue --network bg-lab --env-file .env -e LLM_PROVIDER=fake -e K_R
EVISION=blue -e BANNER=BLUE friend-agent:local

docker run -d --name green --network bg-lab --env-file .env -e LLM_PROVIDER=fake -e K_
REVISION=green -e BANNER=GREEN friend-agent:local
```

여기서 기억할 옵션은 다음과 같습니다.

- `-env-file .env`: 파일에 저장된 환경변수를 컨테이너에 전달합니다.
- `e`: 개별 환경변수를 지정합니다. 같은 변수라면 `e`로 지정한 값이 `-env-file`의 값보다 우선합니다.
- `-network bg-lab`: 두 컨테이너를 기존 Docker 네트워크에 연결합니다.
- `p`가 없음: Blue와 Green의 포트를 호스트에 직접 공개하지 않습니다.

컨테이너 IP가 바뀌었을 수 있으므로 Nginx 설정을 다시 읽게 합니다.

```
docker exec bg-nginx nginx -s reload
```

그리고 Blue의 시작 로그를 확인합니다.

```
docker logs blue 2>&1 | head -5
```

Windows PowerShell에서는 다음처럼 앞의 다섯 줄을 확인할 수 있습니다.

```
docker logs blue 2>&1 | Select-Object -First 5
```

로그에 저장소와 인증 방식이 Supabase로 설정되었다는 정보 및 서버 실행 메시지가 나타나는지 확인합니다. 오류가 있다면 `.env`의 필수 값과 Supabase 인증 설정을 확인하되, 비밀 키를 로그나 채팅에 출력하지 마세요.

7. 실습 ③ 실제로 데이터가 유지되는지 확인하기

이제 설정이 완료되었다고 가정하는 데서 끝내지 않고, 실제로 데이터가 유지되는지 검증합니다.

1. Blue에서 데이터 생성

   `http://localhost:8080`을 열고 게스트 사용자로 시작합니다. 캐릭터를 저장하고 대화와 약속을 하나씩 만듭니다.

2. Supabase에 저장됐는지 확인

   Supabase 대시보드의 Table Editor에서 `messages`와 `reminders` 테이블에 데이터가 생성되었는지 확인합니다. 브라우저에서 보낸 데이터가 외부 데이터베이스에 기록되는지 확인하는 단계입니다.

3. Green으로 전환

   Nginx 설정을 `server green:8080;`으로 바꾸고 `nginx -t` 검사 후 reload합니다. 브라우저를 새로고침해 같은 대화와 약속이 보이는지 확인합니다.

4. 컨테이너 재시작

   `docker restart blue`를 실행하고 Nginx를 reload한 뒤 Blue로 롤백합니다. 기존 대화와 약속이 유지되는지 확인합니다.

5. 다른 브라우저에서 확인

   시크릿 창이나 다른 브라우저에서 서비스를 열어 봅니다. 다른 게스트 사용자로 시작하므로 앞서 만든 대화가 보이지 않는 것이 기대 결과입니다.

결과를 해석하는 방법

실험	기대 결과	의미
Blue에서 Green으로 전환	대화와 약속 유지	두 컨테이너가 공용 DB를 사용
Blue 컨테이너 재시작	대화와 약속 유지	데이터가 컨테이너 메모리에만 있지 않음

실험	기대 결과	의미
다른 브라우저로 접속	앞 사용자의 데이터가 보이지 않음	사용자 식별과 데이터 접근 제한이 동작

이 결과가 확인되면 데이터 저장 구조가 이전보다 개선된 것입니다. 단, 실제로 안전하다고 판단하려면 RLS 정책과 서버의 사용자별 데이터 조회·쓰기 로직도 함께 검증해야 합니다.

8. 새롭게 생기는 위험: 데이터베이스 호환성

외부 데이터베이스를 사용하면 대화가 유지되지만, 새로운 운영 문제가 생깁니다.

Blue와 Green이 같은 테이블을 공유하기 때문입니다.

예를 들어 Green이 기존 `messages` 테이블의 컬럼 이름을 바꾸었는데 Blue는 이전 컬럼 이름을 사용한다면, Green으로 전환했다가 Blue로 롤백했을 때 문제가 생길 수 있습니다.

이를 해결하기 위해 앞서 배운 확장-축소(Expand-and-Contract) 방식을 사용합니다.

1. 기존 버전도 사용할 수 있도록 새 컬럼을 추가합니다.
2. 새 버전이 새 구조를 사용하도록 배포합니다.
3. 기존 버전이 더 이상 해당 구조를 사용하지 않는지 확인합니다.
4. 마지막에 사용하지 않는 옛 구조를 제거합니다.

즉, 블루-그린 배포에서는 코드뿐 아니라 데이터베이스 구조도 두 버전이 함께 사용할 수 있도록 관리해야 합니다.

9. 객관식 퀴즈 정답

퀴즈 1. Secret key와 publishable key의 차이

정답: 2번

Secret key는 RLS를 우회해 높은 권한으로 데이터에 접근할 수 있으므로 서버에서만 사용해야 합니다. Publishable key는 브라우저에서 사용하도록 설계되었지만, 데이터 보호를 위해 RLS와 적절한 접근 정책이 반드시 필요합니다.

퀴즈 2. 데이터베이스 연결 후 새롭게 생기는 위험

정답: 2번

두 버전이 같은 테이블을 공유하기 때문에 한 버전의 테이블 구조 변경이 다른 버전의 실행이나 롤백에 영향을 줄 수 있습니다. 따라서 데이터베이스 변경도 이전 버전과 호환되도록 관리해야 합니다.

10. 실습 종료 및 정리

실습이 끝나면 로컬 컨테이너와 네트워크를 정리할 수 있습니다.

```
docker rm -f blue green bg-nginx
docker network rm bg-lab
```

이 명령을 실행하기 전에 실제로 제거하려는 컨테이너와 네트워크가 맞는지 확인하세요. 이 정리는 로컬 Docker 환경을 대상으로 하며, Supabase 프로젝트와 `.env` 파일은 제거하지 않습니다.

이후 다시 로컬에서 서비스를 실행하면 같은 시험용 데이터베이스를 사용할 수 있습니다. 메모리 저장 방식으로 되돌려야 한다면 교안에 따라 `.env`의 `STORAGE`와 `AUTH` 값을 변경하고, 해당 설정이 반영되도록 서비스를 다시 실행해야 합니다.

11. AI 개발자와 1인 창업가가 기억할 점

AI 서비스에서 사용자의 대화, 요약, 기억, 약속은 단순한 임시 데이터가 아닙니다. 사용자가 서비스를 계속 이용하게 만드는 핵심 자산입니다.

따라서 운영 가능한 AI 서비스를 만들려면 다음 세 가지를 함께 설계해야 합니다.

- 서비스 실행: 컨테이너를 교체하고 재시작할 수 있어야 합니다.
- 데이터 지속성: 컨테이너가 바뀌어도 사용자 데이터가 남아야 합니다.
- 데이터 보안: 각 사용자는 자신의 데이터에만 접근할 수 있어야 합니다.

MSM처럼 사용자의 개인 기록과 이야기를 다루는 서비스라면 데이터가 사라지지 않는 것뿐 아니라, 다른 사용자의 기록이 섞이지 않는 것도 매우 중요합니다.

한 문장 요약

이번 챕터에서는 Supabase와 사용자 인증을 연결해 대화 데이터를 컨테이너 밖에 저장하고, Blue와 Green을 전환하거나 재시작해도 사용자 경험이 이어지는 무상태 서비스의 기반을 마련합니다.

Chapter07

/강의정리 챕터 07 — 고가용성과 무중단 배포

1. 오늘 수업의 목적

오늘 수업의 핵심은 새 버전의 서비스를 배포하거나 서버에 장애가 발생해도 사용자가 서비스를 계속 이용할 수 있게 만드는 방법을 배우는 것입니다.

개발자가 새 버전을 만들었다고 해서 곧바로 기존 서버를 교체하면 위험합니다. 교체하는 순간 요청이 실패하거나, 사용자의 대화가 사라지거나, 새 버전과 데이터베이스가 맞지 않을 수 있기 때문입니다.

오늘은 직접 Blue-Green 배포와 장애 실험을 해 보고, 데이터베이스를 연결해 사용자 데이터가 유지되는 것까지 확인했습니다.

2. 중학생도 이해하는 핵심 개념

Mastering Workloads in Kubernetes with Kueue — part 1: Queues and Priorities | by Michał Żyliński | Google Cloud - Community | Medium

① 고가용성(High Availability)

식당의 계산대가 하나뿐이면 고장 났을 때 모든 손님이 기다려야 합니다. 계산대를 두 개 준비하고, 하나가 고장 나면 다른 쪽으로 손님을 보내면 서비스를 계속할 수 있습니다.

서버도 마찬가지입니다. 이중화, 장애 감지, 장애 조치가 핵심입니다.

Database Blue-Green Deployment: A Practical Guide | Bytebase

② 블루-그린 배포(Blue-Green Deployment)

기존 매장인 Blue를 운영하면서 새 매장인 Green을 미리 준비합니다. Green이 정상인지 확인한 다음 안내판을 바꿔 손님을 Green으로 보냅니다.

문제가 생기면 다시 Blue로 안내할 수 있습니다. 이때 요청을 어느 서버로 보낼지 결정하는 역할이 **로드밸런서(Nginx)**입니다.

Autoscaling architecture - Architecting Cloud Computing Solutions [Book]

③ 무상태 서비스와 외부 데이터베이스

두 매장이 같은 손님의 주문 내역을 알아야 한다면, 각 매장의 직원 기억에만 의존해서는 안 됩니다. 공용 장부에 기록해야 합니다.

서버의 메모리가 아니라 Supabase 같은 외부 데이터베이스에 대화와 약속을 저장하면, 서버가 바뀌어도 사용자 데이터가 유지됩니다.

3. 오늘 직접 실험한 내용과 배운 점

실험	직접 확인한 내용	배운 점
Blue → Green 전환	Nginx 설정을 바꾸고 요청을 전환했습니다.	새 서버가 준비되어 있다면 전환을 빠르게 할 수 있습니다.

실험	직접 확인한 내용	배운 점
롤백	다시 Blue로 요청을 보냈습니다.	문제가 생겼을 때 이전 버전으로 돌아갈 수 있습니다.
가중치 분배	Blue와 Green에 3:1 비율로 요청을 보냈습니다.	새 버전에 일부 요청만 보내는 카나리 배포와 비슷한 원리를 이해할 수 있습니다.
서버 중단	서버가 멈췄을 때 연결 대기 시간에 따라 실패 수가 달라졌습니다.	장애 감지와 타임아웃 설정이 사용자 경험에 영향을 줍니다.
Supabase 연결	서버를 전환해도 기존 대화가 이어졌습니다.	데이터를 서버 외부에 저장하면 서버 교체와 사용자 상태를 분리할 수 있습니다.

특히 기억할 세 가지

첫째, 전환 성공과 무중단은 같은 뜻이 아닙니다.

설정을 바꾸고 새 서버로 요청을 보내는 데 성공했더라도, 전환 중 일부 요청이 실패할 수 있습니다. 반대로 실험에서 실패가 0건이었다고 해서 모든 상황에서 무중단이 보장되는 것도 아닙니다.

둘째, 서버가 살아 있는 것과 사용자의 경험이 유지되는 것은 다릅니다.

서버가 정상적으로 응답하더라도 사용자의 대화 기록이 사라지면 서비스는 제대로 작동한다고 보기 어렵습니다. 서비스 가용성과 데이터 연속성을 각각 검증해야 합니다.

셋째, 데이터베이스를 공유하면 새로운 위험도 생깁니다.

Blue와 Green이 같은 데이터베이스를 사용하면 데이터가 이어진다는 장점이 있습니다. 하지만 한 버전에서 데이터베이스 구조를 바꾸면 다른 버전이나 롤백이 작동하지 않을 수 있습니다. 따라서 배포할 때는 데이터베이스의 이전 버전과 새 버전이 함께 작동할 수 있는지 확인해야 합니다.

4. 주관식 퀴즈 모범 답안

질문

섹션 1에서 예상했던 새 버전 전환 시의 사용자 영향을 오늘 직접 한 실험과 비교해, 맞았던 점과 새로 알게 된 점을 적어 보세요.

모범 답안

처음에는 새 버전으로 전환하는 순간 서비스가 잠시 중단되거나 사용자의 요청이 실패할 수 있다고 생각했습니다. 실제 실험에서는 Blue에서 Green으로 전환하고 다시 롤백하는 과정에서 요청 실패가 0건인 경우도 확인했습니다. 따라서 새 서버를 미리 준비하고 로드밸런서의 설정을 바꾸면 서비스 중단을 줄일 수 있다는 예상은 맞았습니다.

하지만 서버 하나를 강제로 중단했을 때는 연결 대기 시간에 따라 요청 실패 수가 달라졌습니다. 이를 통해 서버에 장애가 발생했다는 사실을 감지하고 다른 서버로 전환하는 데에도 시간이 걸리며, 타임아웃 설정이 중요하다는 것을 새롭게 알게 되었습니다.

또한 서버를 전환하자 기존 대화가 사라지는 문제도 확인했습니다. 서버 내부 메모리에만 데이터를 저장하면 다른 서버가 그 정보를 알 수 없기 때문입니다. 이후 Supabase와 익명 로그인을 연결해 데이터를 외부에 저장하자 Blue와 Green을 전환해도 대화가 이어졌습니다.

결과적으로 무중단 배포를 위해서는 서버 전환만 잘하는 것으로 충분하지 않고, 장애 감지와 복구, 데이터의 연속성, 데이터베이스 호환성까지 함께 설계해야 한다는 점을 배웠습니다.

5. AI 개발자와 1인 창업가에게 왜 중요할까요?

AI 서비스를 운영한다고 가정해 보겠습니다. 사용자가 AI와 30분 동안 대화하며 중요한 정보를 입력했는데, 새 버전을 배포하자 대화가 사라진다면 사용자는 서비스를 신뢰하기 어렵습니다.

오늘 배운 기술은 이런 문제를 예방하는 기반입니다.

- AI 에이전트 운영: 서버를 교체해도 대화 기록과 약속을 유지합니다.
- 서비스 개선: 새 버전을 일부 사용자에게 먼저 적용하고 결과를 확인할 수 있습니다.
- 장애 대응: 서버가 멈췄을 때 감지하고 정상 서버로 요청을 전환할 수 있습니다.
- 사업 신뢰도: 고객의 데이터와 서비스 이용 경험을 안정적으로 보호할 수 있습니다.

특히 AI 서비스를 만드는 창업가에게는 기능을 구현하는 능력뿐 아니라, 고객이 서비스를 계속 믿고 사용할 수 있게 운영하는 능력도 중요합니다.

6. 꼭 기억할 핵심 정리

고가용성 = 이중화 + 장애 감지 + 장애 조치

서버 하나에 문제가 생겨도 서비스가 계속 작동하도록 합니다.

무중단 배포 = 안전한 전환 + 장애 대비

전환뿐 아니라 헬스체크, 타임아웃, 롤백도 중요합니다.

무상태 서비스 + 외부 데이터베이스

서버와 사용자 데이터를 분리해 서버를 바꿔도 대화가 유지되도록 합니다.

공유 데이터베이스의 호환성

새 버전을 배포할 때 이전 버전과 롤백도 고려해야 합니다.

7. 한 문장 요약

고가용성 서비스는 서버를 여러 개 준비하는 것에서 끝나지 않고, 장애가 발생했을 때 안전하게 전환하며 사용자 데이터와 경험까지 유지하도록 설계하는 서비스입니다.